

CO1 – AT1: Data Interpretation

Question 1: Employee Salary Analysis

A CSV file (employees.csv) contains employee details. Task: read the CSV file, display employees earning more than ₹50,000, and calculate the average salary department-wise.

Python Code

```
import pandas as pd

# Read the CSV file
df = pd.read_csv("employees.csv")
print("Full Employee Dataset:")
print(df)

# Employees earning more than 50,000
print("\nEmployees earning more than Rs.50,000:")
high_earners = df[df["Salary"] > 50000]
print(high_earners)

# Average salary department-wise
print("\nDepartment-wise Average Salary:")
avg_salary = df.groupby("Department")["Salary"].mean().round(2)
print(avg_salary)
```

Output

```
●●● Output — Employee Salary Analysis

Full Employee Dataset:
  EmployeeID      Name Department  Salary
0         101  Amit Sharma         IT   55000
1         102  Priya Singh         HR   42000
2         103  Rahul Verma         IT   61000
3         104  Sneha Patel    Finance   48000
4         105  Vikram Rao         HR   39000
5         106  Neha Gupta    Finance   53000
6         107  Arjun Nair         IT   47000
7         108  Kavya Iyer    Finance   58000

Employees earning more than Rs.50,000:
  EmployeeID      Name Department  Salary
0         101  Amit Sharma         IT   55000
2         103  Rahul Verma         IT   61000
5         106  Neha Gupta    Finance   53000
7         108  Kavya Iyer    Finance   58000

Department-wise Average Salary:
Department
Finance    53000.00
HR         40500.00
IT         54333.33
Name: Salary, dtype: float64
```

Question 2: Missing Data Handling

Given a CSV file containing student marks with missing values. Task: identify missing values, replace missing marks with the column average, and display the cleaned dataset.

Python Code

```
import pandas as pd

# Read dataset with missing values
df = pd.read_csv("students_marks.csv")
print("Original Dataset:")
print(df)

# Identify missing values
print("\nMissing Values in Each Column:")
print(df.isnull().sum())

# Replace missing marks with column average
col_avg = df["Marks"].mean()
df["Marks"] = df["Marks"].fillna(round(col_avg, 2))
```

```
# Display the cleaned dataset
print("\nCleaned Dataset (missing marks replaced with average):")
print(df)
```

Output

```

Original Dataset:
  StudentID  Name Subject Marks
0         1  Ananya  Maths  85.0
1         2   Rohan  Maths   NaN
2         3   Isha  Maths  78.0
3         4   Karan  Maths  92.0
4         5   Diya  Maths   NaN
5         6  Manav  Maths  65.0

Missing Values in Each Column:
StudentID    0
Name         0
Subject      0
Marks        2
dtype: int64

Cleaned Dataset (missing marks replaced with average):
  StudentID  Name Subject Marks
0         1  Ananya  Maths  85.0
1         2   Rohan  Maths  80.0
2         3   Isha  Maths  78.0
3         4   Karan  Maths  92.0
4         5   Diya  Maths  80.0
5         6  Manav  Maths  65.0

```

Question 4: Sorting Dataset

Given sales data. Task: sort products based on sales amount and display the top 10 highest-selling products.

Python Code

```
import pandas as pd

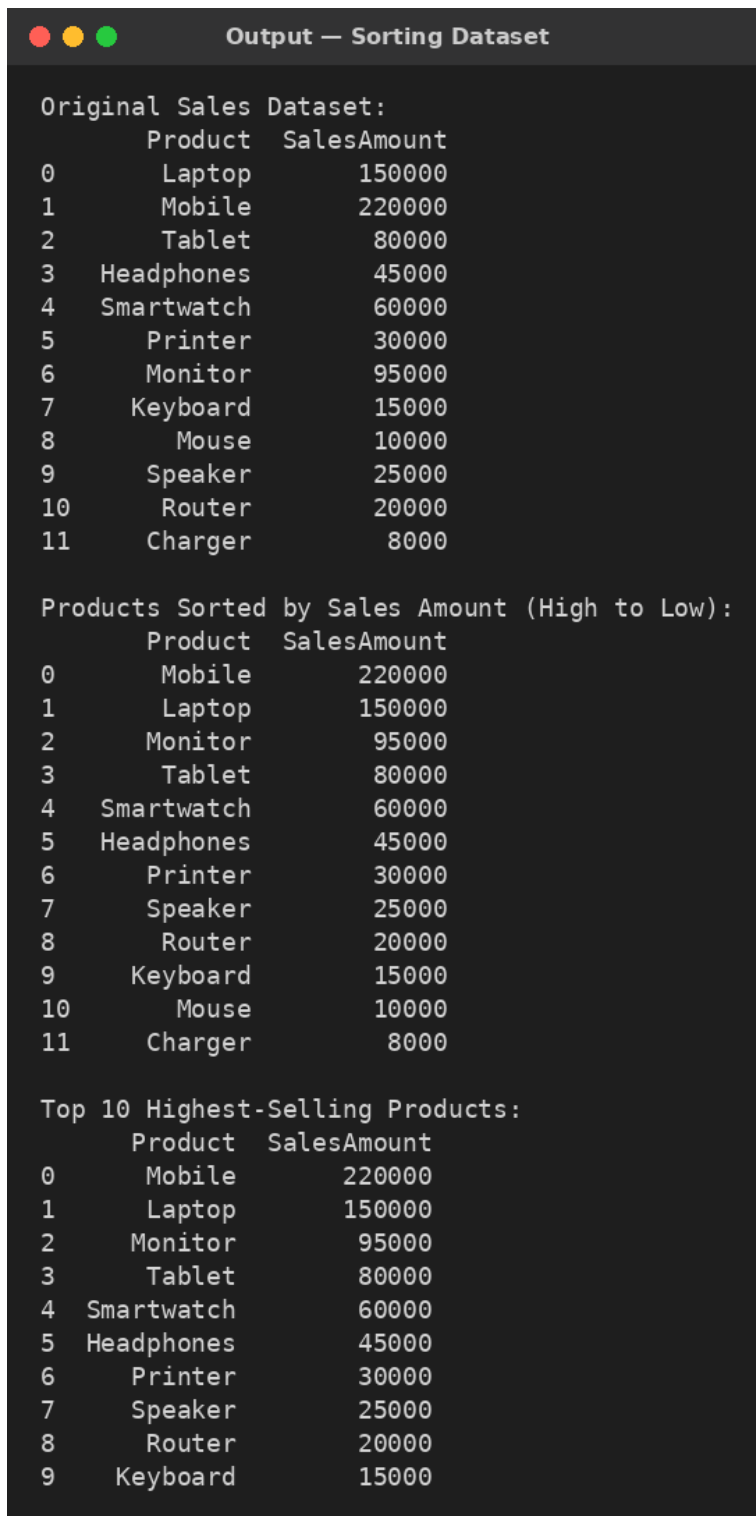
# Read the sales dataset
df = pd.read_csv("sales.csv")
print("Original Sales Dataset:")
print(df)

# Sort products based on sales amount (descending)
sorted_df = df.sort_values(by="SalesAmount", ascending=False).reset_index(drop=True)
print("\nProducts Sorted by Sales Amount (High to Low):")
print(sorted_df)

# Display top 10 highest-selling products
```

```
print("\nTop 10 Highest-Selling Products:")
print(sorted_df.head(10))
```

Output



Output — Sorting Dataset

Original Sales Dataset:

	Product	SalesAmount
0	Laptop	150000
1	Mobile	220000
2	Tablet	80000
3	Headphones	45000
4	Smartwatch	60000
5	Printer	30000
6	Monitor	95000
7	Keyboard	15000
8	Mouse	10000
9	Speaker	25000
10	Router	20000
11	Charger	8000

Products Sorted by Sales Amount (High to Low):

	Product	SalesAmount
0	Mobile	220000
1	Laptop	150000
2	Monitor	95000
3	Tablet	80000
4	Smartwatch	60000
5	Headphones	45000
6	Printer	30000
7	Speaker	25000
8	Router	20000
9	Keyboard	15000
10	Mouse	10000
11	Charger	8000

Top 10 Highest-Selling Products:

	Product	SalesAmount
0	Mobile	220000
1	Laptop	150000
2	Monitor	95000
3	Tablet	80000
4	Smartwatch	60000
5	Headphones	45000
6	Printer	30000
7	Speaker	25000
8	Router	20000
9	Keyboard	15000

Question 7: Data Transformation

A product dataset stores prices in USD. Task: convert prices into INR and add a new column called "Price_INR".

Python Code

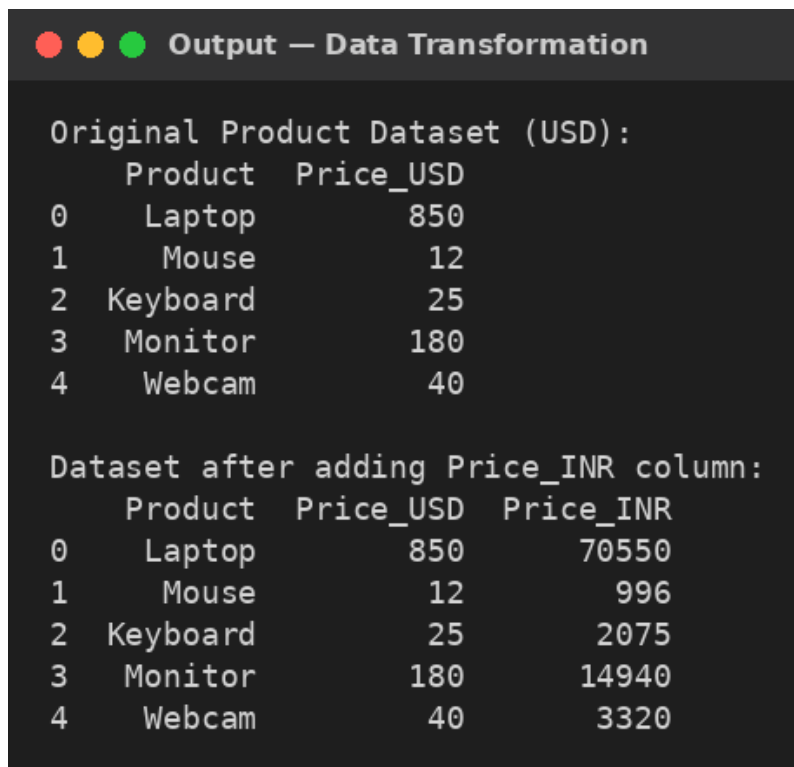
```
import pandas as pd

# Product dataset with prices in USD
data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor", "Webcam"],
    "Price_USD": [850, 12, 25, 180, 40]
}
df = pd.DataFrame(data)
print("Original Product Dataset (USD):")
print(df)

# Convert prices into INR (1 USD = 83 INR)
usd_to_inr = 83
df["Price_INR"] = df["Price_USD"] * usd_to_inr

print("\nDataset after adding Price_INR column:")
print(df)
```

Output



The terminal window displays the output of the Python code. It shows the original dataset with 5 products and their prices in USD, followed by the transformed dataset with an additional 'Price_INR' column calculated by multiplying the USD price by 83.

```
Output — Data Transformation

Original Product Dataset (USD):
   Product  Price_USD
0  Laptop         850
1  Mouse          12
2 Keyboard         25
3  Monitor        180
4  Webcam          40

Dataset after adding Price_INR column:
   Product  Price_USD  Price_INR
0  Laptop         850        70550
1  Mouse          12         996
2 Keyboard         25        2075
3  Monitor        180       14940
4  Webcam          40        3320
```

Question 9: Grouping Data

Given employee information. Task: group employees by department and calculate total salary, average salary, and employee count.

Python Code

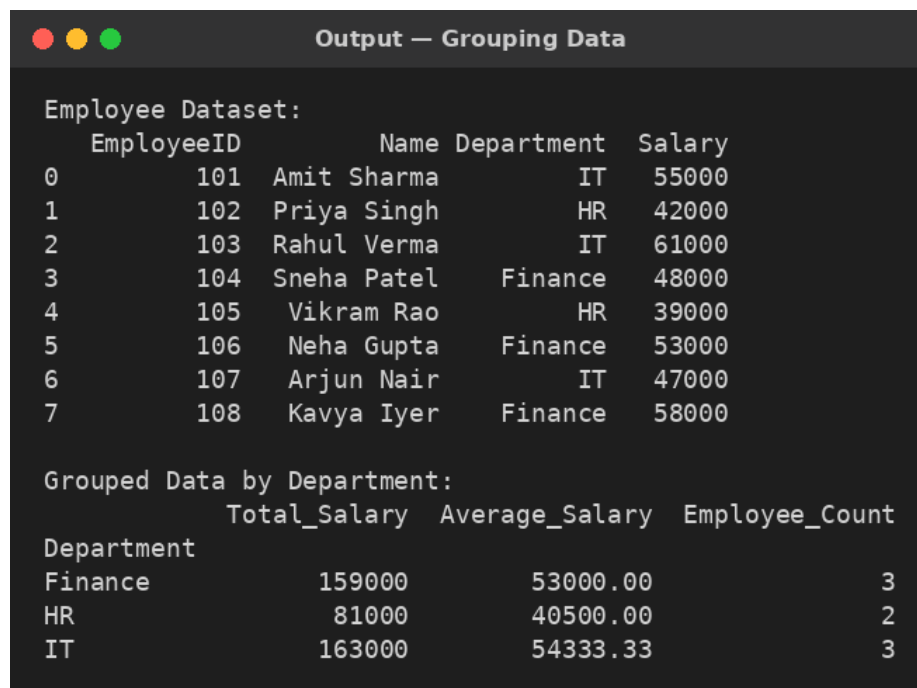
```
import pandas as pd

# Reuse employee dataset
df = pd.read_csv("employees.csv")
print("Employee Dataset:")
print(df)

# Group employees by department
grouped = df.groupby("Department")["Salary"].agg(
    Total_Salary="sum",
    Average_Salary="mean",
    Employee_Count="count"
).round(2)

print("\nGrouped Data by Department:")
print(grouped)
```

Output



Employee Dataset:

	EmployeeID	Name	Department	Salary
0	101	Amit Sharma	IT	55000
1	102	Priya Singh	HR	42000
2	103	Rahul Verma	IT	61000
3	104	Sneha Patel	Finance	48000
4	105	Vikram Rao	HR	39000
5	106	Neha Gupta	Finance	53000
6	107	Arjun Nair	IT	47000
7	108	Kavya Iyer	Finance	58000

Grouped Data by Department:

	Total_Salary	Average_Salary	Employee_Count
Department			
Finance	159000	53000.00	3
HR	81000	40500.00	2
IT	163000	54333.33	3

Question 11: Read JSON Data

Given a JSON file containing student information. Task: read JSON data, display student names, and find the topper.

Python Code

```
import json

# Read JSON data
with open("students.json", "r") as f:
    students = json.load(f)

print("Student Data:")
for s in students:
```

```

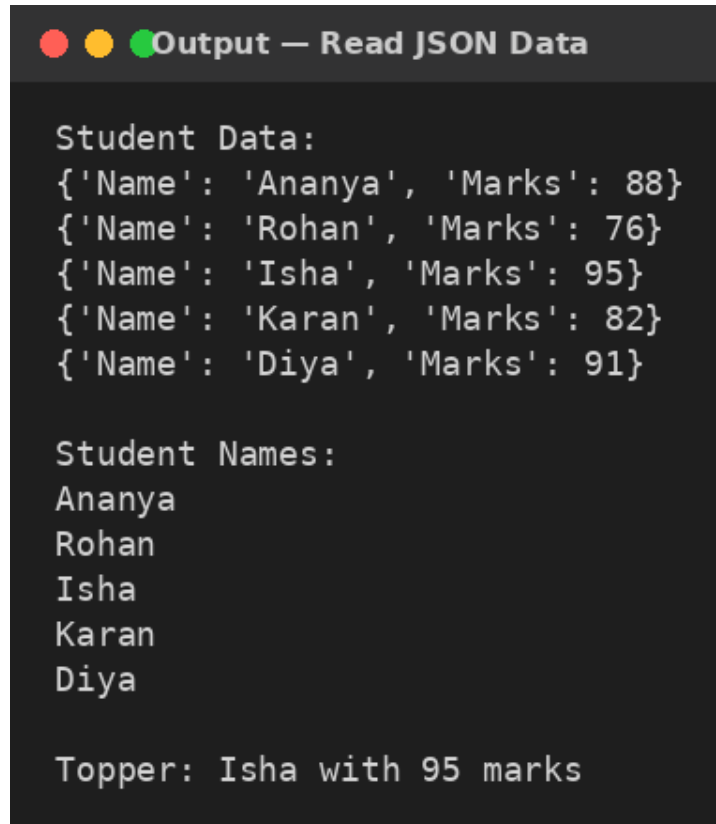
print(s)

# Display student names
print("\nStudent Names:")
for s in students:
    print(s["Name"])

# Find the topper
topper = max(students, key=lambda s: s["Marks"])
print(f"\nTopper: {topper['Name']} with {topper['Marks']} marks")

```

Output



```

Student Data:
{'Name': 'Ananya', 'Marks': 88}
{'Name': 'Rohan', 'Marks': 76}
{'Name': 'Isha', 'Marks': 95}
{'Name': 'Karan', 'Marks': 82}
{'Name': 'Diya', 'Marks': 91}

Student Names:
Ananya
Rohan
Isha
Karan
Diya

Topper: Isha with 95 marks

```

Question 14: Read XML File

Given employee.xml. Task: read XML data, display employee names, and count total employees.

Python Code

```

import xml.etree.ElementTree as ET

# Read XML data
tree = ET.parse("employee.xml")
root = tree.getroot()

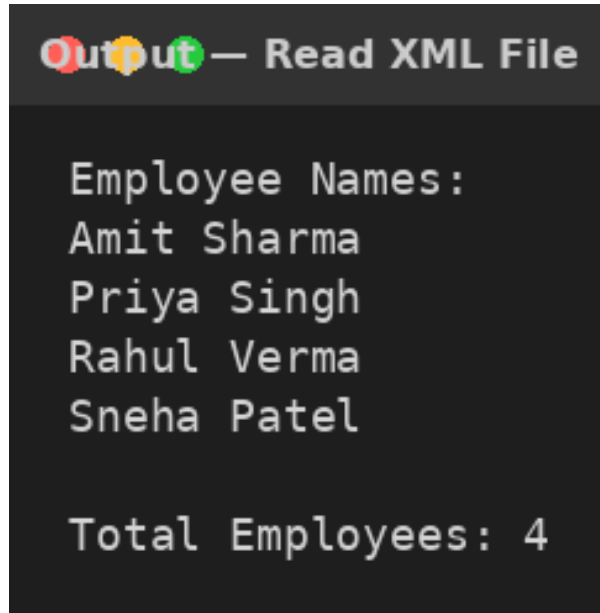
# Display employee names
print("Employee Names:")
names = []
for emp in root.findall("Employee"):
    name = emp.find("Name").text

```

```
names.append(name)
print(name)

# Count total employees
print(f"\nTotal Employees: {len(names)}")
```

Output

A terminal window with a dark background. The title bar reads "Output — Read XML File". The output text is as follows:

```
Employee Names:
Amit Sharma
Priya Singh
Rahul Verma
Sneha Patel

Total Employees: 4
```

Question 16: XML Data Analysis

Given library.xml. Task: display total books, books written after 2020, and books costing more than ₹500.

Python Code

```
import xml.etree.ElementTree as ET

# Read library XML data
tree = ET.parse("library.xml")
root = tree.getroot()

books = []
for b in root.findall("Book"):
    books.append({
        "Title": b.find("Title").text,
        "Year": int(b.find("Year").text),
        "Price": int(b.find("Price").text)
    })

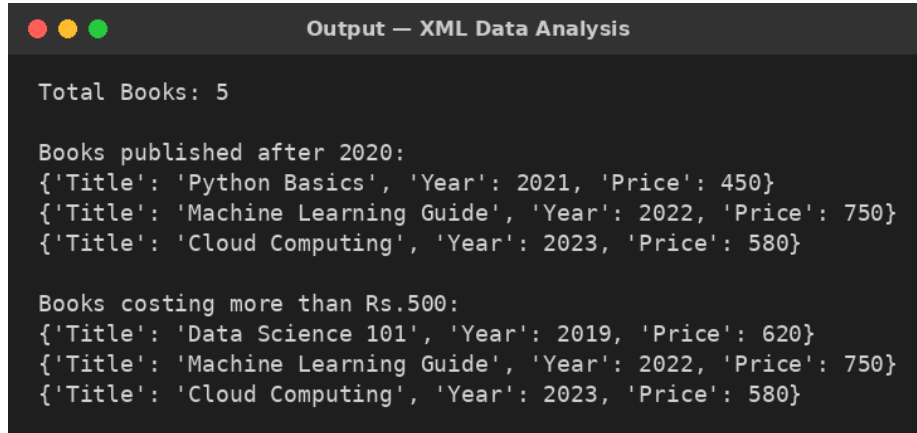
# Total books
print(f"Total Books: {len(books)}")

# Books written after 2020
after_2020 = [b for b in books if b["Year"] > 2020]
print("\nBooks published after 2020:")
for b in after_2020:
    print(b)
```



```
# Books costing more than 500
costly = [b for b in books if b["Price"] > 500]
print("\nBooks costing more than Rs.500:")
for b in costly:
    print(b)
```

Output



```
Output — XML Data Analysis

Total Books: 5

Books published after 2020:
{'Title': 'Python Basics', 'Year': 2021, 'Price': 450}
{'Title': 'Machine Learning Guide', 'Year': 2022, 'Price': 750}
{'Title': 'Cloud Computing', 'Year': 2023, 'Price': 580}

Books costing more than Rs.500:
{'Title': 'Data Science 101', 'Year': 2019, 'Price': 620}
{'Title': 'Machine Learning Guide', 'Year': 2022, 'Price': 750}
{'Title': 'Cloud Computing', 'Year': 2023, 'Price': 580}
```

Question 18: Dictionary-Based Analysis

Store employee information using dictionaries. Task: display the highest salary employee, lowest salary employee, and average salary.

Python Code

```
# Store employee information using dictionaries
employees = {
    "Amit Sharma": 55000,
    "Priya Singh": 42000,
    "Rahul Verma": 61000,
    "Sneha Patel": 48000,
    "Vikram Rao": 39000
}

print("Employee Salary Dictionary:")
for name, salary in employees.items():
    print(f"{name}: Rs.{salary}")

# Highest salary employee
highest = max(employees, key=employees.get)
print(f"\nHighest Salary Employee: {highest} (Rs.{employees[highest]})")

# Lowest salary employee
lowest = min(employees, key=employees.get)
print(f"Lowest Salary Employee: {lowest} (Rs.{employees[lowest]})")

# Average salary
average = sum(employees.values()) / len(employees)
print(f"Average Salary: Rs.{average:.2f}")
```

Output



Output — Dictionary-Based Analysis

Employee Salary Dictionary:

Amit Sharma: Rs.55000

Priya Singh: Rs.42000

Rahul Verma: Rs.61000

Sneha Patel: Rs.48000

Vikram Rao: Rs.39000

Highest Salary Employee: Rahul Verma (Rs.61000)

Lowest Salary Employee: Vikram Rao (Rs.39000)

Average Salary: Rs.49000.00